

华东师范大学软件工程学院实验报告

姓名：高珩博

学号：10255101445

实验编号：Lab2

实验名称：BombLab

一、实验目的

要求掌握逆向分析能力，汇编语言理解，gdb 与 objdump 等调试工具的使用。

二、实验内容与实验步骤

实验主要内容：在 linux 环境下，用 objdump 工具对可执行文件 bomb 反编译，并阅读汇编代码，找到正确输入，从而实现“拆弹”。全部过程中共 6 个主“炸弹”+1 个隐藏关卡，全部解出即实验成功。

实验步骤：

1. 通过 `nm bomb | grep phase` 命令查找目标文件 bomb 中含 phase 字样的所有符号表，从中找到我们所需要的函数名。
2. 通过 objdump 的命令，将每个 phase 的主要函数反编译成汇编语言。
3. 阅读分析汇编语言，解出对应输入
4. 最后输入正确“密钥”，实现拆弹。

三、实验过程与分析

首先，先运行 `nm bomb | grep phase` 命令

```
boyu@LAPTOP-0A8VIRCU:/mnt/a/bomb$ nm bomb | grep phase
0000000004012f6 T invalid_phase
000000000400ee0 T phase_1
000000000400efc T phase_2
000000000400f43 T phase_3
00000000040100c T phase_4
000000000401062 T phase_5
0000000004010f4 T phase_6
0000000004015c4 T phase_defused
000000000401242 T secret_phase
```

可发现，主要的函数名叫做 phase_1—6，以及一个 secret_phase，可知共七个炸弹，我们一个一个拆。

1. Phase1:

先用 objdump 反编译

```
boyu@LAPTOP-0A8VIRCU:/mnt/a/bomb$ objdump -d bomb | sed -n '/<phase_1>:/,/^$/p'
000000000400ee0 <phase_1>:
400ee0: 48 83 ec 08      sub    $0x8,%rsp
400ee4: be 00 24 40 00   mov    $0x402400,%esi
400ee9: e8 4a 04 00 00   call   401338 <strings_not_equal>
400eee: 85 c0           test   %eax,%eax
400ef0: 74 05          je     400ef7 <phase_1+0x17>
400ef2: e8 43 05 00 00   call   40143a <explode_bomb>
400ef7: 48 83 c4 08      add    $0x8,%rsp
400efb: c3             ret
```

分析发现 phase_1 函数调用了函数 strings_not_equal，从该函数名可知，是用于比较两个字符串是否相等。在该函数被调用之前，程序将用户输入的字符串作为第一个参数传递给了 strings_not_equal 函数。可知，该函数的第一个参数是用户所认定的密码。

根据汇编代码中的操作，可以发现目标字符串保存在了内存地址 0x402400 处，需要将该地址中的字符串作为比较的第二个参数传递给 strings_not_equal 函数，从而得知，对应的密钥字符串就存在 0x402400 处，直接用在 gdb 下访问即可。

```
(gdb) x/s 0x402400
0x402400: "Border relations with Canada have never been better."
```

如此，将这串字符串 *Border relations with Canada have never been better.* 输入即可拆弹。

```
boyu@LAPTOP-0A8VIRCUC:/mnt/a/bomb$ ./bomb
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Border relations with Canada have never been better.
Phase 1 defused. How about the next one?
```

2. Phase2:

```
boyu@LAPTOP-0A8VIRCUC:/mnt/a/bomb$ objdump -d bomb | sed -n '/<phase_2>:/,/^$/p'
000000000400efc <phase_2>:
400efc: 55          push    %rbp
400efd: 53          push    %rbx
400efe: 48 83 ec 28  sub    $0x28,%rsp
400ef0: 48 89 e6     mov     %rsp,%rsi
400ef5: e8 52 05 00 00 call   40145c <read_six_numbers>
400efa: 83 3c 24 01  cmpl   $0x1,(%rsp)
400efb: 74 20       je      400f30 <phase_2+0x34>
400ef8: e8 25 05 00 00 call   40143a <explode_bomb>
400ef5: eb 19       jmp     400f30 <phase_2+0x34>
400ef7: 8b 43 fc     mov     -0x4(%rbx),%eax
400efa: 01 c0       add     %eax,%eax
400efc: 39 03       cmp     %eax,(%rbx)
400efe: 74 05       je      400f25 <phase_2+0x29>
400ef8: e8 15 05 00 00 call   40143a <explode_bomb>
400ef5: 48 83 c3 04  add    $0x4,%rbx
400ef2: 48 39 eb     cmp     %rbp,%rbx
400ef2: 75 e9       jne     400f17 <phase_2+0x1b>
400ef2: eb 0c       jmp     400f3c <phase_2+0x40>
400ef3: 48 8d 5c 24 04 lea     0x4(%rsp),%rbx
400ef5: 48 8d 6c 24 18 lea     0x18(%rsp),%rbp
400ef3: eb db       jmp     400f17 <phase_2+0x1b>
400ef3: 48 83 c4 28  add    $0x28,%rsp
400ef4: 5b         pop     %rbx
400ef5: 5d         pop     %rbp
400ef6: c3         ret
```

分析发现 phase_2 调用了 read_six_numbers 函数，由函数名可知本题需要输入 6 个数字。现在我们寻找一下六个数字的关系。

开头在栈上分配了 0x28 也就是 40 字节的空间，然后栈顶地址传输给 read_six_numbers 函数，会读取六个数字。

成功读取了数字之后，回到 400efa，比较的是 (%rsp) 和 1，即第一个数字必须为 1，否则引爆炸弹，输入 1 以后跳转到 400f30，执行：rbx = &num[1];rbp = &nums[6]，然后跳转到 400f17，执行：%eax=&rbx-0x4，即 eax = nums[1-1]，然后 eax*=2，比较 nums[1] 和 eax，不相等直接引爆炸弹，相等则跳转到 400f25，先对 %rbx 存储的地址加 4，也就是 rbx=num[1++]，检查 %rbp 和 %rbx 是否相等，相等跳转到 400f3c 即函数末尾，否则跳转回 400f17。

那么整理一下逻辑，可以发现这其实就是一个循环函数，如果后一位不等于前一位的二倍，就会爆炸，再结合首项为 1，不难发现正确结果是：

1 2 4 8 16 32

输入结果：

```
1 2 4 8 16 32
That's number 2. Keep going!
```

进入第三关。

3. Phase3:

```
000000000400f43 <phase_3>:
400f43: 48 83 ec 18  sub    $0x18,%rsp
400f47: 48 8d 4c 24 0c lea     0xc(%rsp),%rcx
400f4c: 48 8d 54 24 08 lea     0x8(%rsp),%rdx
400f51: be cf 25 40 00 mov     $0x4025cf,%esi
400f56: b8 00 00 00 00 mov     $0x0,%eax
400f5b: e8 90 fc ff ff call   400bf0 <__isoc99_sscanf@plt>
400f60: 83 f8 01     cmp     $0x1,%eax
400f63: 7f 05       jg      400f6a <phase_3+0x27>
400f65: e8 d0 04 00 00 call   40143a <explode_bomb>
400f6a: 83 7c 24 08 07 cmpl    $0x7,0x8(%rsp)
400f6f: 77 3c       ja      400fad <phase_3+0x6a>
400f71: 8b 44 24 08  mov     0x8(%rsp),%eax
400f75: ff 24 c5 70 24 40 00 jmp     *0x402470(,%rax,8)
400f7c: b8 cf 00 00 00 mov     $0xcf,%eax
400f81: eb 3b       jmp     400f8e <phase_3+0x7b>
400f83: b8 c3 02 00 00 mov     $0x2c3,%eax
400f88: eb 34       jmp     400f8e <phase_3+0x7b>
400f8a: b8 00 01 00 00 mov     $0x100,%eax
400f8f: eb 2d       jmp     400f8e <phase_3+0x7b>
400f91: b8 85 01 00 00 mov     $0x185,%eax
400f96: eb 26       jmp     400f8e <phase_3+0x7b>
400f98: b8 ce 00 00 00 mov     $0xce,%eax
400f9d: eb 1f       jmp     400f8e <phase_3+0x7b>
400fa4: eb 18       jmp     400f8e <phase_3+0x7b>
400fa6: b8 47 01 00 00 mov     $0x147,%eax
400fab: eb 11       jmp     400f8e <phase_3+0x7b>
400fad: e8 88 04 00 00 call   40143a <explode_bomb>
400fb2: b8 00 00 00 00 mov     $0x0,%eax
400fb7: eb 05       jmp     400f8e <phase_3+0x7b>
```

代码开头同样是在栈上分配 24 字节空间，读取字符串后再用 sscanf 进行读取，查看其地

址 0x4025cf 后可知:

```
(gdb) x/s 0x4025cf
0x4025cf: "%d %d"
```

我们知道这道题需要输入两个整数，并再次观察逻辑可以发现 `cmpl $0x7,0x8(%rsp)` 该句是在检查第一个输入值是否在 0-7 范围内，如果 `x>7` 就会跳转到 400fad 即爆炸，可知第一个值范围在 0-7。

再观察，发现很多条都执行了 `jmp` 跳转指令，故而访问基地址 0x402470，去查看跳转表。

```
(gdb) x/16a 0x402470
0x402470: 0x400f7c <phase_3+57> 0x400fb9 <phase_3+118>
0x402480: 0x400f83 <phase_3+64> 0x400f8a <phase_3+71>
0x402490: 0x400f91 <phase_3+78> 0x400f98 <phase_3+85>
0x4024a0: 0x400f9f <phase_3+92> 0x400fa6 <phase_3+99>
0x4024b0: <array.3449>: 0x737265697564616d 0x6c7962767466666e
0x4024c0: 0x7420756f79206f53 0x756f79206b6e6968
0x4024d0: 0x6f7473206e616320 0x6f62206568742070
0x4024e0: 0x206874697720626d 0x202c632d6c727463
```

可以发现，对于不同的 `x` 值，有不同的跳转，回到原函数发现不同的 `x` 对应不同的期望值，从而形成了一个映射表：

```
0->0xcf=207
1->0x2c3=707
2->0x100=256
3->0x185=389
4->0xce=206
5->0x2aa=682
6->0x147=327
7->0x137=311
```

所以只需要选一组对应的就好了
输入结果

```
That's number 2. Keep going!
0 207
Halfway there!
```

进入下一关。

4. Phase4:

```
0000000040100c <phase_4>:
40100c: 48 83 ec 18      sub    $0x18,%rsp
401010: 48 8d 4c 24 0c    lea    0xc(%rsp),%rcx
401015: 48 8d 54 24 08    lea    0x8(%rsp),%rdx
40101a: be cf 25 40 00    mov    $0x4025cf,%esi
40101f: b8 00 00 00 00    mov    $0x0,%eax
401024: e8 c7 fb ff ff    call   400bf0 <__isoc99_sscanf@plt>
401029: 83 f8 02         cmp    $0x2,%eax
40102c: 75 07            jne    401035 <phase_4+0x29>
40102e: 83 7c 24 08 0e    cmpl   $0xe,0x8(%rsp)
401033: 76 05            jbe    40103a <phase_4+0x2e>
401035: e8 00 04 00 00    call   40143a <explode_bomb>
40103a: ba 0e 00 00 00    mov    $0xe,%edx
40103f: be 00 00 00 00    mov    $0x0,%esi
401044: 8b 7c 24 08      mov    0x8(%rsp),%edi
401048: e8 81 ff ff ff    call   400fce <func4>
40104d: 85 c0            test   %eax,%eax
40104f: 75 07            jne    401058 <phase_4+0x4c>
401051: 83 7c 24 0c 00    cmpl   $0x0,0xc(%rsp)
401056: 74 05            je     40105d <phase_4+0x51>
401058: e8 dd 03 00 00    call   40143a <explode_bomb>
40105d: 48 83 c4 18      add    $0x18,%rsp
401061: c3              ret
```

函数开头在栈上分配了 24 字节的空间，然后调用 `sscanf` 对字符串进行格式化，查看格式化内容得到：

```
(gdb) x/s 0x4025cf
0x4025cf: "%d %d"
```

可知本题也需要两个整数

观察汇编代码发现 `cmpl $0xe,0x8(%rsp)`，要确保 `x<=14`，否则爆炸，如果 `x<=14` 则跳

转到 40103a, 将 edx 赋值为 14, esi 赋值为 0, edi 赋值为 x, 跳转到函数 func4。

对于函数 func4, 将 edx 赋值给 eax, eax 再减去 esi, eax 再赋给 ecx, ecx 右移 31 位然后与 eax 相加, 结果储存在 eax 中, eax 右移一位, 和 rsi 相加后存储到 ecx 中, 将 ecx 与 x 进行比较, 若 ecx <= x 则跳到 400ff2: 将 eax 赋值为 0, 如果 ecx >= x, 跳到 401007, 即函数返回; 否则 esi=rcx+1, 再次调用自身进行递归, 同时对递归结果进行处理 rax=rax*2+1; 如果 ecx > x 则将 rcx-1 后赋给 edx, 调用自身递归, 最终返回值 eax*2。

返回到 phase_4 函数中, 程序将 %eax 中的值与 0 进行比较, eax 不为零引爆炸弹, eax=0 则进行下一步比较, 如果 y=0 则跳转到末尾, 否则引爆。

从而可以知道, x, y 均为 0。

输入结果。

```
0 0
So you got that one. Try this one.
```

成功进入下一关

5. Phase5:

```
000000000401062: <phase_5>:
401062: 53                                push    %rbx
401063: 48 83 ec 20                       sub     $0x20,%rsp
401067: 48 89 fb                          mov     %rdi,%rbx
40106a: 64 48 8b 04 25 28 00             mov     %fs:0x28,%rax
401071: 00 00
401073: 48 89 44 24 18                   mov     %rax,0x18(%rsp)
401078: 31 c0                             xor     %eax,%eax
40107a: e8 9c 02 00 00                   call    40131b <string_length>
40107f: 83 f8 06                         cmp     $0x6,%eax
401082: 74 4e                             je      4010d2 <phase_5+0x70>
401084: e8 b1 03 00 00                   call    40143a <explode_bomb>
401089: eb 47                             jmp     4010d2 <phase_5+0x70>
40108b: 0f b6 0c 03                       movzbl  (%rbx,%rax,1),%ecx
40108f: 88 0c 24                         mov     %cl,(%rsp)
401092: 48 8b 14 24                       mov     (%rsp),%rdx
401096: 83 e2 0f                         and     $0xf,%edx
401099: 0f b6 92 b0 24 40 00             movzbl  0x4024b0(%rdx),%edx
4010a0: 88 54 04 10                       mov     %dl,0x10(%rsp,%rax,1)
4010a4: 48 83 c0 01                       add     $0x1,%rax
4010a8: 48 83 f8 06                       cmp     $0x6,%rax
4010ac: 75 dd                             jne     40108b <phase_5+0x29>
4010ae: c6 44 24 16 00                   movb    $0x0,0x16(%rsp)
4010b3: be 5e 24 40 00                   mov     $0x40245e,%esi
4010b8: 48 8d 7c 24 10                   lea     0x10(%rsp),%rdi
4010bd: e8 76 02 00 00                   call    401338 <strings_not_equal>
4010c2: 85 c0                             test    %eax,%eax
4010c4: 74 13                             je      4010d9 <phase_5+0x77>
4010c6: e8 6f 03 00 00                   call    40143a <explode_bomb>
4010cb: 0f 1f 44 00 00                   nopl    0x0(%rax,%rax,1)
4010d0: eb 07                             jmp     4010d9 <phase_5+0x77>
4010d2: b8 00 00 00 00                   mov     $0x0,%eax
4010d7: eb b2                             jmp     40108b <phase_5+0x29>
4010d9: 48 8b 44 24 18                   mov     0x18(%rsp),%rax
4010de: 64 48 33 04 25 28 00             xor     %fs:0x28,%rax
4010e5: 00 00
4010e7: 74 05                             je      4010ee <phase_5+0x8c>
4010e9: e8 42 fa ff ff                   call    400b30 <_stack_chk_fail@plt>
4010ee: 48 83 c4 20                       add     $0x20,%rsp
4010f2: 5b                                pop     %rbx
4010f3: c3                                ret
```

首先可以看到调用了 string_length 函数, 并和 6 进行了 cmp, 如果长度不为 6, 则跳转爆炸, 可知本题密钥是六位字符串。

再观察汇编代码, 发现存在一个循环, xor %eax,%eax 在设置循环计数器, mov %rdi,%rbx, 将字符串指针保存到 %rbx 中, 从而开始循环, 对单个字符操作完毕后, 环计数器 rax 加一然后跳转回 40108b 继续循环, 一共处理六个字符。

循环结束后从 0x40245e 中读取目标字符串放到 %esi, 调用 strings_not_equal 进行比较, 若不一致则爆炸, 于是只需要让最初的字符串经过循环中的字符变换, 最终和地址 0x40245e 的字符串保持一致即可过关。

先看循环逻辑:

and \$0xf,%edx, 与 0xf 取与运算, 即取低四位, 然后将该数字索引在 0x4024b0 中进行检索, 并入栈形成新的字符串。

所以只需要从目标字符串倒推即可获得结果。

目标字符串是:

```
(gdb) x/s 0x40245e
0x40245e: "flyers"
```

进行逆推得到索引为 9, 15, 14, 5, 6, 7, 查找 ASCII 表得到后四位为 1001, 1111, 1110, 0101, 0110, 0111 的小写字母, 得到字符串 ionefg, 输入即

```
ionefg
Good work! On to the next...
```

6. Phase6:

```

0000000004010f4 <phase_6>:
4010f4: 41 56                push    %r14
4010f6: 41 55                push    %r13
4010f8: 41 54                push    %r12
4010fa: 55                  push    %rbp
4010fb: 53                  push    %rbx
4010fc: 48 83 ec 50         sub     $0x50,%rsp
401100: 49 89 e5            mov     %rsp,%r13
401103: 48 89 e6            mov     %rsp,%rsi
401106: e8 51 03 00 00     call   40145c <read_six_numbers>
40110b: 49 89 e6            mov     %rsp,%r14
40110e: 41 bc 00 00 00 00   mov     $0x0,%r12d
401114: 4c 89 ed            mov     %r13,%rbp
401117: 41 8b 45 00         mov     0x0(%r13),%eax
40111b: 83 e8 01            sub     $0x1,%eax
40111e: 83 f8 05            cmp     $0x5,%eax
401121: 76 05              jbe     401128 <phase_6+0x34>
401123: e8 12 03 00 00     call   40143a <explode_bomb>
401128: 41 83 c4 01         add     $0x1,%r12d
40112c: 41 83 fc 06         cmp     $0x6,%r12d
401130: 74 21              je      401153 <phase_6+0x5f>
401132: 44 89 e3            mov     %r12d,%ebx
401135: 48 63 c3            movslq  %ebx,%rax
401138: 8b 04 84            mov     (%rsp,%rax,4),%eax
40113b: 39 45 00            cmp     %eax,0x0(%rbp)
40113e: 75 05              jne     401145 <phase_6+0x51>
401140: e8 f5 02 00 00     call   40143a <explode_bomb>
401145: 83 c3 01            add     $0x1,%ebx
401148: 83 fb 05            cmp     $0x5,%ebx
40114b: 7e e8              jle     401135 <phase_6+0x41>
40114d: 49 83 c5 04         add     $0x4,%r13
401151: eb c1              jmp     401114 <phase_6+0x20>
401153: 48 8d 74 24 18     lea     0x18(%rsp),%rsi
401158: 4c 89 f0            mov     %r14,%rax
40115b: b9 07 00 00 00     mov     $0x7,%ecx
401160: 89 ca              mov     %ecx,%edx
401162: 2b 10              sub     (%rax),%edx
401164: 89 10              mov     %edx,(%rax)
401166: 48 83 c0 04         add     $0x4,%rax
40116a: 48 39 f0            cmp     %rsi,%rax
40116d: 75 f1              jne     401160 <phase_6+0x6c>
40116f: be 00 00 00 00     mov     $0x0,%esi
401174: eb 21              jmp     401197 <phase_6+0xa3>
401176: 48 8b 52 08         mov     0x8(%rdx),%rdx
40117a: 83 c0 01            add     $0x1,%eax
40117d: 39 c8              cmp     %ecx,%eax
40117f: 75 f5              jne     401176 <phase_6+0x82>
401181: eb 05              jmp     401188 <phase_6+0x94>
401183: ba d0 32 60 00     mov     $0x6032d0,%edx
401188: 48 89 54 74 20     mov     %rdx,0x20(%rsp,%rsi,2)
40118d: 48 83 c6 04         add     $0x4,%rsi
401191: 48 83 fe 18         cmp     $0x18,%rsi
401181: eb 05              jmp     401188 <phase_6+0x94>
401183: ba d0 32 60 00     mov     $0x6032d0,%edx
401188: 48 89 54 74 20     mov     %rdx,0x20(%rsp,%rsi,2)
40118d: 48 83 c6 04         add     $0x4,%rsi
401191: 48 83 fe 18         cmp     $0x18,%rsi
401195: 74 14              je      4011ab <phase_6+0xb7>
401197: 8b 0c 34            mov     (%rsp,%rsi,1),%ecx
40119a: 83 f9 01            cmp     $0x1,%ecx
40119d: 7e e4              jle     401183 <phase_6+0x8f>
40119f: b8 01 00 00 00     mov     $0x1,%eax
4011a4: ba d0 32 60 00     mov     $0x6032d0,%edx
4011a9: eb cb              jmp     401176 <phase_6+0x82>
4011ab: 48 8b 5c 24 20     mov     0x20(%rsp),%rbx
4011b0: 48 8d 44 24 28     lea     0x28(%rsp),%rax
4011b5: 48 8d 74 24 50     lea     0x50(%rsp),%rsi
4011ba: 48 89 d9            mov     %rbx,%rcx
4011bd: 48 b1 10            mov     (%rax),%rdx
4011c0: 48 89 51 08         mov     %rdx,0x8(%rcx)
4011c4: 48 83 c0 08         add     $0x8,%rax
4011c8: 48 39 f0            cmp     %rsi,%rax
4011cb: 74 05              je      4011d2 <phase_6+0xde>
4011cd: 48 89 d1            mov     %rdx,%rcx
4011d0: eb eb              jmp     4011bd <phase_6+0xc9>
4011d2: 48 c7 42 08 00 00 00 movq    $0x0,0x8(%rdx)
4011d9: 00
4011da: bd 05 00 00 00     mov     $0x5,%ebp
4011df: 48 8b 43 08         mov     0x8(%rbx),%rax
4011e3: 8b 00              mov     (%rax),%eax
4011e5: 39 03              cmp     %eax,(%rbx)
4011e7: 7d 05              jge     4011ee <phase_6+0xfa>
4011e9: e8 4c 02 00 00     call   40143a <explode_bomb>
4011ee: 48 8b 5b 08         mov     0x8(%rbx),%rbx
4011f2: 83 ed 01            sub     $0x1,%ebp
4011f5: 75 e8              jne     4011df <phase_6+0xeb>
4011f7: 48 83 c4 50         add     $0x50,%rsp
4011fb: 5b                pop     %rbx
4011fc: 5d                pop     %rbp
4011fd: 41 5c              pop     %r12
4011ff: 41 5d              pop     %r13
401201: 41 5e              pop     %r14
401203: c3                ret

```

发现调用了 read_six_numbers 函数，意味着会读取六个整数，并通过观察汇编语言可以发现是对这六个整数进行循环操作，数字-1 后与 5 比较，即数字如果大于 6 就会爆

炸，跳转到 401128 以后先是对循环计数器进行检测，然后跳转到 401153，对数字进行处理，`numbers[i] = 7 - numbers[i]`。

后面发现是一段指针操作，可以看出来是一个链表，访问地址发现

```
(gdb) x/64x 0x6032d0
0x6032d0 <node1>: 0x0000014c 0x00000001 0x006032e0 0x00000000
0x6032e0 <node2>: 0x000000a3 0x00000002 0x006032f0 0x00000000
0x6032f0 <node3>: 0x00000039c 0x00000003 0x00603300 0x00000000
0x603300 <node4>: 0x0000002b3 0x00000004 0x00603310 0x00000000
0x603310 <node5>: 0x000001dd 0x00000005 0x00603320 0x00000000
0x603320 <node6>: 0x000001bb 0x00000006 0x00000000 0x00000000
0x603330: 0x00000000 0x00000000 0x00000000 0x00000000
0x603340 <host_table>: 0x00402629 0x00000000 0x00402643 0x00000000
0x603350 <host_table+16>: 0x0040265d 0x00000000 0x00000000 0x00000000
0x603360 <host_table+32>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603370 <host_table+48>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603380 <host_table+64>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603390 <host_table+80>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6033a0 <host_table+96>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6033b0 <host_table+112>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6033c0 <host_table+128>: 0x00000000 0x00000000 0x00000000 0x00000000
```

可以得出链表节点关系

0x6032d0→0x6032e0→0x6032f0→0x603300→0x603310→0x603320

再观察后续函数操作，可以得出得到后续代码的功能是根据输入的数字进行节点的重新选取，然后按顺序对六个节点进行重新连接，连接顺序如下：

`node[3]>node[4]>node[5]>node[6]>node[1]>node[2]`

但这是经过了 `7-numbers[i]` 的顺序，所以本输入的顺序应该是 4, 3, 2, 1, 6, 5 输入即可通过

```
4 3 2 1 6 5
Congratulations! You've defused the bomb!
boyu@LAPTOP-0A8VIRCU:/mnt/a/bomb$
```

7. Phase7:

隐藏关卡，从 c 语言源码的注释和最开始我们查询的全部函数可以看到，存在隐藏关卡。那我们先看对应的函数

```
000000000401242 <secret_phase>:
401242: 53                push    %rbx
401243: e8 56 02 00 00    call   40149e <read_line>
401248: ba 0a 00 00 00    mov     $0xa,%edx
40124d: be 00 00 00 00    mov     $0x0,%esi
401252: 48 89 c7          mov     %rax,%rdi
401255: e8 76 f9 ff ff    call   400bd0 <strtol@plt>
40125a: 48 89 c3          mov     %rax,%rbx
40125d: 8d 40 ff          lea     -0x1(%rax),%eax
401260: 3d e8 03 00 00    cmp     $0x3e8,%eax
401265: 76 05            jbe     40126c <secret_phase+0x2a>
401267: e8 ce 01 00 00    call   40143a <explode_bomb>
40126c: 89 de            mov     %ebx,%esi
40126e: bf f0 30 60 00    mov     $0x6030f0,%edi
401273: e8 8c ff ff ff    call   401204 <fun7>
401278: 83 f8 02          cmp     $0x2,%eax
40127b: 74 05            je      401282 <secret_phase+0x40>
40127d: e8 b8 01 00 00    call   40143a <explode_bomb>
401282: bf 38 24 40 00    mov     $0x402438,%edi
401287: e8 84 f8 ff ff    call   400b10 <puts@plt>
40128c: e8 33 03 00 00    call   4015c4 <phase_defused>
401291: 5b                pop     %rbx
401292: c3                ret

000000000401204 <fun7>:
401204: 48 83 ec 08       sub     $0x8,%rsp
401208: 48 85 ff          test    %rdi,%rdi
40120b: 74 2b            je      401238 <fun7+0x34>
40120d: 8b 17            mov     (%rdi),%edx
40120f: 39 f2            cmp     %esi,%edx
401211: 7e 0d            jle     401220 <fun7+0x1c>
401213: 48 8b 7f 08       mov     0x8(%rdi),%rdi
401217: e8 e8 ff ff ff    call   401204 <fun7>
40121c: 01 c0            add     %eax,%eax
40121e: eb 1d            jmp     40123d <fun7+0x39>
401220: b8 00 00 00 00    mov     $0x0,%eax
401225: 39 f2            cmp     %esi,%edx
401227: 74 14            je      40123d <fun7+0x39>
401229: 48 8b 7f 10       mov     0x10(%rdi),%rdi
40122d: e8 d2 ff ff ff    call   401204 <fun7>
401232: 8d 44 00 01       lea     0x1(%rax,%rax,1),%eax
401236: eb 05            jmp     40123d <fun7+0x39>
401238: b8 ff ff ff ff    mov     $0xffffffff,%eax
40123d: 48 83 c4 08       add     $0x8,%rsp
401241: c3                ret
```

```

0000000004015c4 <phase_defused>:
4015c4: 48 83 ec 78      sub    $0x78,%rsp
4015c8: 64 48 8b 04 25 28 00 mov    %fs:0x28,%rax
4015cf: 00 00
4015d1: 48 89 44 24 68    mov    %rax,0x68(%rsp)
4015d6: 31 c9            xor    %eax,%eax
4015d8: 83 3d 81 21 20 00 06 cmpl    $0x6,0x202181(%rip) # 603760 <num_input_strings>
4015df: 75 5e            jne    40163f <phase_defused+0x7b>
4015e1: 4c 8d 44 24 10    lea    0x10(%rsp),%r8
4015e6: 48 8d 4c 24 0c    lea    0xc(%rsp),%rcx
4015eb: 48 8d 54 24 08    lea    0x8(%rsp),%rdx
4015f0: be 19 26 40 00    mov    $0x402619,%esi
4015f5: bf 70 39 60 00    mov    $0x603870,%edi
4015fa: e8 f1 f5 ff ff    call   400bf0 <_isoc99_sscanf@plt>
4015ff: 83 f8 03         cmp    $0x3,%eax
401602: 75 31            jne    401635 <phase_defused+0x71>
401604: be 22 26 40 00    mov    $0x402622,%esi
401609: 48 8d 7c 24 10    lea    0x10(%rsp),%rdi
40160e: e8 25 fd ff ff    call   401338 <strings_not_equal>
401613: 85 c0            test   %eax,%eax
401615: 75 1e            jne    401635 <phase_defused+0x71>
401617: bf f8 24 40 00    mov    $0x4024f8,%edi
40161c: e8 ef f4 ff ff    call   400b10 <puts@plt>
401621: bf 20 25 40 00    mov    $0x402520,%edi
401626: e8 e5 f4 ff ff    call   400b10 <puts@plt>
40162b: b8 00 00 00 00    mov    $0x0,%eax
401630: e8 0d fc ff ff    call   401242 <secret_phase>
401635: bf 58 25 40 00    mov    $0x402558,%edi
40163a: e8 d1 f4 ff ff    call   400b10 <puts@plt>
40163f: 48 8b 44 24 68    mov    0x68(%rsp),%rax
401644: 64 48 33 04 25 28 00 xor    %fs:0x28,%rax
40164b: 00 00
40164d: 74 05            je     401654 <phase_defused+0x90>
40164f: e8 dc f4 ff ff    call   400b30 <__stack_chk_fail@plt>
401654: 48 83 c4 78      add    $0x78,%rsp
401658: c3              ret
401659: 90              nop
40165a: 90              nop
40165b: 90              nop
40165c: 90              nop
40165d: 90              nop
40165e: 90              nop
40165f: 90              nop

```

可以发现该段代码中调用了隐藏关卡的函数，首先在 4015d8 中调用函数对输入过的字符串数量进行统计，如果不是 6，一定无法进入隐藏关，然后要读取字符串，与 0x402622 的值进行比较，判断%eax 的值，如果是 3 则能够进入 secret_phase，调用其中几个内存地址 0x402619, 0x603870, 0x402622 的值为：

```

(gdb) x/s 0x402619
0x402619: "%d %d %s"
(gdb) x/s 0x603870
0x603870 <input_strings+240>: ""
(gdb) x/s 0x402622
0x402622: "DrEvil"
(gdb)

```

可知需要在一些位置输入两个整数一个字符串，全部匹配即可进入隐藏关卡。我们发现这个 0x603870 地址值为空，故回到调用 sscanf 处设置断点再次进行调试：

```

(gdb) print (char*) 0x603870
$4 = 0x603870 <input_strings+240> "0 0"

```

可知两个数字是 0 0

并且可知读取的是 phase4 的内容 猜测在 hase_4 的输入中加上“DrEvil”即可在最后开启隐藏关

```

Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Border relations with Canada have never been better.
Phase 1 defused. How about the next one?
1 2 4 8 16 32
That's number 2. Keep going!
0 207
Halfway there!
0 0 DrEvil
So you got that one. Try this one.
ioneftg
Good work! On to the next...
4 3 2 1 6 5
Curses, you've found the secret phase!
But finding it and solving it are quite different...

```

接下来我们分析隐藏关函数

对 fun7 进行分析，不难看出和第六关的函数有些相似，经过分析可以得出，这是一个含有两个指针的结构体，前面的 7 个结构体的两个指针都是有值的，它们指向其他的结构体。而最后 8 个结构体的指针全为 0，结合数据结构的知识可以知道，这是一个二叉树。

由 phase_secret 函数可以知道，初始值经过一系列运算最终值必须要等于 2，否则就会爆炸。

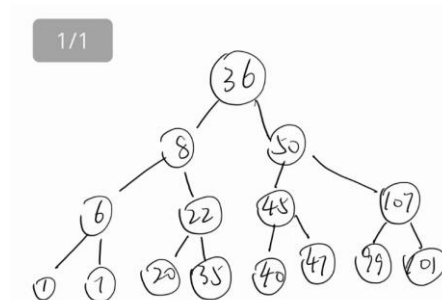
而从 fun7 中可以看出运算逻辑，为表述方便，我将其转换为 C 语言代码：

```
int func7(Type *p, int input){
    if(p == NULL) return -1;
    if(&p <= input){
        if(&p == input) return 0;
        else{
            p = p + 0x10;
            int n = func7(p, input);
            return 2 * n + 1;
        }
    }
    else{
        p = p + 0x8;
        int n = func7(p, input);
        return 2 * n;}
}
```

先查询 0x6030f0 记录每个节点的值：

```
(gdb) x/150x 0x6030f0
0x6030f0: 0x00000024 0x00000000 0x00603110 0x00000000
0x603100: <n1>: 0x00603130 0x00000000 0x00000000 0x00000000
0x603110: <n21>: 0x00000008 0x00000000 0x00603190 0x00000000
0x603120: <n21+16>: 0x00603150 0x00000000 0x00000000 0x00000000
0x603130: <n22>: 0x00000032 0x00000000 0x00603170 0x00000000
0x603140: <n22+16>: 0x006031b0 0x00000000 0x00000000 0x00000000
0x603150: <n32>: 0x00000016 0x00000000 0x00603270 0x00000000
0x603160: <n32+16>: 0x00603230 0x00000000 0x00000000 0x00000000
0x603170: <n33>: 0x0000002d 0x00000000 0x006031d0 0x00000000
0x603180: <n33+16>: 0x00603290 0x00000000 0x00000000 0x00000000
0x603190: <n31>: 0x00000006 0x00000000 0x006031f0 0x00000000
0x6031a0: <n31+16>: 0x00603250 0x00000000 0x00000000 0x00000000
0x6031b0: <n34>: 0x0000006b 0x00000000 0x00603210 0x00000000
0x6031c0: <n34+16>: 0x006032b0 0x00000000 0x00000000 0x00000000
0x6031d0: <n45>: 0x00000028 0x00000000 0x00000000 0x00000000
0x6031e0: <n45+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6031f0: <n41>: 0x00000001 0x00000000 0x00000000 0x00000000
0x603200: <n41+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603210: <n47>: 0x00000063 0x00000000 0x00000000 0x00000000
0x603220: <n47+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603230: <n44>: 0x00000023 0x00000000 0x00000000 0x00000000
0x603240: <n44+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603250: <n42>: 0x00000007 0x00000000 0x00000000 0x00000000
0x603260: <n42+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603270: <n43>: 0x00000014 0x00000000 0x00000000 0x00000000
0x603280: <n43+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x603290: <n46>: 0x0000002f 0x00000000 0x00000000 0x00000000
0x6032a0: <n46+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6032b0: <n48>: 0x000000e9 0x00000000 0x00000000 0x00000000
0x6032c0: <n48+16>: 0x00000000 0x00000000 0x00000000 0x00000000
0x6032d0: <node1>: 0x0000014c 0x00000001 0x006032e0 0x00000000
0x6032e0: <node2>: 0x000000a8 0x00000002 0x006032f0 0x00000000
0x6032f0: <node3>: 0x00000039c 0x00000003 0x00603300 0x00000000
0x603300: <node4>: 0x000002b3 0x00000004 0x00603310 0x00000000
0x603310: <node5>: 0x000001dd 0x00000005 0x00603320 0x00000000
0x603320: <node6>: 0x000001bb 0x00000006 0x00000000 0x00000000
0x603330: 0x00000000 0x00000000 0x00000000 0x00000000
```

画出图来是：



从子节点倒推，有两种情况：

1. 父节点在左侧，小于 $0*2+1$ ，祖父节点在右侧，大于 $1*2=2$ ，回到起点，可知值为 22
2. 父节点在右侧，大于 $0*2=0$ ，祖父节点在左侧，小于 $0*2+1=1$ ，曾祖父节点在右侧，大于 $1*2=2$ ，回到起点，可知值为 20；

所以密钥为 20 或者 22

输入

```
COU8L9fUJ9f70U2; 10N,16 qeLn2eq fme powp;
MOM; 10N,16 qeLn2eq fme zecwef zfa8e;
50
```


成功!!!

四、实验结果总结

整个实验流程说起来很简单，但是具体的操作以及分析很复杂很复杂，尤其是后面几个实验，汇编语言的阅读难度太高了，只能一点一点慢慢啃，有任何不懂不清楚的地方就借助大语言模型询问学习，包括命令行的指令，比如调用 `objdump` 并返回汇编内容 的这条命令行，也是在询问 ai 之后才学习到的。

做完整个实验，让我对汇编语言有了更清晰的认知，对 `gdb` 等调试工具的掌握更加熟练，让我收获颇丰。

五、附录（源代码）

见正文截图